PINTOP TECHNOLOGIES — QUALITY ASSURANCE ENGINEERING

Week 1 and Week 2: Instructor Read-Aloud Talking Notes

Case study: ShopDesk Quality Gate

How to use: Everything under **SAY** is written to be spoken directly. **DEMO** means share your screen and execute the commands. **ASK** means invite a short answer before continuing. Each session has approximately 50–60 minutes of speaking and guided practice. The official handbook allocates two-hour live sessions; use the remaining time for independent labs, questions, and assignment guidance. Notes align with handbook pp. 109–161, while the setup guide is pp. 5–108.

Before the first class: Install Node 20, Git and VS Code. Unzip `ShopDesk_Complete_Course_Codebase.zip`, open the `shopdesk-course` folder, run `npm run setup`, then in separate terminals run `cd api && npm start` and `cd web && npm run dev`. The QA terminal stays available for exercises. The app uses http://localhost:5173 and the API uses http://localhost:4000.

WEEK 1 — CLASS 1: Quality Concepts and Test Strategy

Handbook: Module 1. **Teaching objective:** Students can distinguish QA/QC, verification/validation, risk, test levels, severity/priority, and draft a strategy.

00:00–00:07 — Welcome and the ShopDesk mission

SAY: “Good evening, everyone, and welcome to Quality Assurance Engineering at Pintop Technologies. Before we open any test framework, I want us to answer a much bigger question. Imagine a company wants to release its online store tonight. The developer says, ‘The website opens on my laptop.’ The manager says, ‘The design looks good.’ The customer says, ‘I want my order to go through and my money to be correct.’ Who is right? They are asking different questions. Our job as quality engineers is to gather evidence that tells us whether it is safe to release, and to explain our recommendation clearly.

“Throughout this course, we will test an example online shop called ShopDesk. Customers can see products, search, add items to a cart, register, log in, place orders, and view order history. We did not hire you to design this store from scratch. We are treating it as the application under test. Its backend uses Node, Express and SQLite. Its interface uses React. Our work lives mainly in the `qa` folder. By Week 5, our tests and evidence will form a Quality Gate: an agreed checkpoint before software can be released.”

ASK: “If a page opens but the customer is charged twice, is that a quality product? Why not?” Let two students answer.

00:07–00:17 — Quality, QA, QC and testing

SAY: “Quality means the product meets its stated requirements and is fit for what people actually need. Two things can be true at once: the developer followed the specification correctly, and the product is still frustrating or unsafe. QA stands for Quality Assurance. QA is preventive. It is about improving the process so mistakes are less likely to enter our product. Example: before developers write checkout, the team agrees that all currency values are integers in kobo and all order totals must be calculated on the server.

“QC means Quality Control. QC checks the product that already exists. Example: I run a test and compare the resulting order total to the expected total. Testing is one important QC activity: I enter data, perform an action and compare actual results against expectations. Think of QA as preventing an exam question from being written wrongly, and QC as marking the completed exam. Both matter.

“For ShopDesk, imagine buying two USB-C hubs at twenty-five thousand naira each. The correct total is fifty thousand naira. In storage that is five million kobo. If a developer accidentally saves fifty thousand kobo, the interface may look fine until the financial records are checked. Quality assurance asks for a clear money rule early. Quality control proves whether the finished software obeys it.”

ASK: “A team adds an automated test requirement to pull requests. Is that QA or QC?” Answer: adding the requirement is QA process design; executing the tests and evaluating results is QC.

00:17–00:24 — Verification, validation and shifting left

SAY: “Verification asks, ‘Are we building it right?’ It checks conformance to documented rules. If the requirement says prices must be integer kobo, we can inspect the API and assert exact integer totals. Validation asks, ‘Are we building the right thing?’ Can a shopper actually search, check out and understand what happened, including on a small phone? We need both. A system can meet all written rules and still solve the wrong problem.

“Shift-left means we begin quality work early, during requirements and design, instead of waiting until the product is completed. A missing security requirement spotted today may take ten minutes to discuss. The same omission found after users have lost access or money can lead to expensive incidents. The popular one–ten–one-hundred cost model is a teaching illustration, not a guaranteed formula, but the principle is sound: prevent avoidable rework before it multiplies. Shift-right matters too: production monitoring, rollback plans, and user feedback keep us learning after release.”

00:24–00:34 — Risk ranking and the test pyramid

SAY: “We cannot test every possible click and input. We prioritise. We score risk as likelihood times impact, often each between one and five. Checkout money might score four for likelihood and five for impact. That gives twenty. A banner colour issue may score three times one, which is three. Where would you spend your limited time first? Money and authentication.

“We also choose the correct testing level. Unit tests check one function, such as the money converter; they are fast and precise. API tests check an endpoint, such as order creation. Integration tests verify several real pieces work together, such as an order and its database transaction. End-to-end tests run through a browser like a real customer. Use many cheap, reliable tests lower down and a smaller number of essential full-journey tests at the top. A whole browser is an unnecessarily expensive place to prove that two plus two equals four.

“ShopDesk has five QA test folders: `unit`, `component`, `api`, `integration` and `e2e`. The aim is to put an assertion at the cheapest level that still proves the behaviour we care about.”

DEMO: Open `qa/test-plans/strategy.md`. Read out R-01, R-02 and R-07 and compare their mitigation. Show `api/src/money.js` without changing it.

00:34–00:43 — Strategy, release gates, severity and priority

SAY: “A test strategy is our long-term agreement on quality. It says what we cover, what is excluded, what environments we use, what risks matter, and when a release should be blocked. A test plan is narrower: it assigns the actual work for a particular release or sprint. If we say ‘everything must work’, we have not set a measurable standard. If we say ‘critical checkout tests must pass in the required browsers, and no unresolved high-impact security defect may ship’, that is a useful condition.

“Severity and priority are not the same. Severity measures how badly the software is affected. Priority measures how urgently the business wants a fix. If customer A can see customer B’s orders, severity is critical and priority is urgent. A typo on a widely seen checkout button may have low technical severity but still have high priority because it damages trust. Bug titles must communicate behaviour, not frustration.”

ASK: “A rarely used admin export crashes. Is the severity necessarily low just because the feature is rarely used?” Answer: no; technical impact and business urgency differ.

00:43–00:52 — SDLC, STLC and practical exercise

SAY: “The Software Development Life Cycle covers the product journey: requirements, design, implementation, testing, deployment and maintenance. The Software Testing Life Cycle is our quality workflow: understand requirements, plan tests, design cases, prepare data and environment, execute tests, report defects and close with evidence. In modern teams these activities overlap rather than waiting for a rigid phase to finish.

“Now create your first professional artefact. Go to `qa/test-plans/strategy.md`. In pairs, identify one additional money risk, one security risk, and one mobile usability risk. Give each a likelihood from one to five, an impact from one to five, and the product of the two. Decide which test level can best detect each risk. The number is a discussion aid, not an objective truth.”

CLOSE: “Today you learned why a green screen is not enough. Next class we will turn risks into individual test cases, explore ShopDesk deliberately, and write a defect report that a developer can reproduce without calling us.”

WEEK 1 — CLASS 2: Manual Test Cases, Exploratory Testing and Defect Reporting

Handbook: Module 2. **Teaching objective:** Students can write clear cases, choose boundaries, run a charter, and document a defect.

00:00–00:08 — Review and the anatomy of a test case

SAY: “Welcome back. Last class we asked what makes ShopDesk safe to release. Today we turn those decisions into evidence. A test case is a repeatable recipe. If I hand it to somebody in another city, they should be able to follow the same steps and reach the same expected outcome. A good test case has an ID, descriptive title, linked requirement or risk, preconditions, test data, steps, expected result, actual result and execution status. We do not mark a test ‘Pass’ merely because we wrote it. We mark Pass only after running it and seeing the expected behaviour.

“Open `qa/test-plans/cases/catalog.md`. Look at TC-AUTH-002. It says to register an email that already exists. Our API should answer 409 and `email_taken`. Notice the expected result is observable. ‘It should work’ is not an expected result.”

00:08–00:20 — Positive, negative, partitions and boundaries

SAY: “Positive testing checks valid input. Negative testing checks invalid input and error handling. Both are necessary. If a registration form accepts valid emails but also allows an empty email, a happy-path test will miss the defect.

“Equivalence partitioning means grouping input values that we expect to behave the same way. For a password rule requiring at least eight characters, one short password and one valid-length password give us useful representatives. Boundary Value Analysis, or BVA, tests values just below, exactly at and just above an edge. For eight characters, that means seven, eight and nine. If a requirement really has an upper bound of sixty-four, we also try sixty-three, sixty-four and sixty-five.

“Be careful: the live ShopDesk API in our provided app only enforces a minimum password length of eight; the stricter eight-to-sixty-four rule appears in a separate Week 2 practice validator. We must not silently assume those two are the same specification. A good QA engineer asks which contract is authoritative.”

ASK: “If we test order quantities one, two and one hundred, what other useful case do we need?” Answer: zero, negative, fractions, and quantity above available stock.

00:20–00:32 — Live manual API demonstration

SAY: “Let us stop theorising and see real responses. Open two terminal tabs. In the first, start the API. In the second, ask the API for its health and then for products. A URL that works is useful evidence, but an exact HTTP status and body tell us much more.”

DEMO: Run each separately from the `shopdesk-course` root:

```
cd api
npm start
```

New terminal:

```
curl -i http://localhost:4000/api/health
curl -i 'http://localhost:4000/api/products?search=desk'
curl -i http://localhost:4000/api/products/9999
```

SAY: “The health route should say status ok. The search should return matching products. A missing product should give 404, with an error code we can assert in Week 3. This is manual API testing: we learn the contract before automating it.”

DEMO: Open http://localhost:5173. Search `desk`, clear search, open a USB-C hub, set quantity to 2, add to cart. Verify ■`50,000.00` and `data-total-kobo=5000000` if inspecting developer tools. Then try checkout while signed out and observe login redirect.

00:32–00:41 — Exploratory testing and charters

SAY: “A scripted case tests something we already decided to check. Exploratory testing is when we learn, design and execute while investigating. That does not mean random clicking. We give ourselves a charter: ‘Explore cart quantity and totals, using changes around available stock, to discover incorrect money or quantity behaviour.’ Then we use a timebox, take notes, and do a short debrief.

“Session-based Test Management tracks how long we explored, how long blockers interrupted us, what went off-charter, and which areas we covered. Open `qa/test-plans/exploratory/charter-template.md`. Start a 10-minute mini-session now. Try adding and removing the same product, changing quantity, checking an empty cart, using the back button, and refreshing. Record observations, not guesses.”

00:41–00:51 — Defect reporting and lifecycle

SAY: “Suppose we find a problem. A bug report is not the sentence ‘checkout is broken’. It is a work order. It should identify the exact build and environment, preconditions, minimal numbered reproduction steps, expected result, actual result, severity, priority and evidence. State only observed facts under Actual. Put theories in notes. Capture screenshots, request/response evidence or a test result. Reproduce it twice before calling it consistent.

“Bugs move through statuses: New, Assigned, In Progress, Fixed, Ready for Retest, Verified and Closed. If the fix fails verification, reopen it with fresh evidence. Retesting the original bug is not enough: also check nearby behaviours affected by the change. That is regression testing.”

DEMO: Open `qa/test-plans/bugs/BUG-template.md`. Write an illustrative report titled ‘Cart total does not update after item removal’, but label it **hypothetical** unless you have reproduced it. Show where exact evidence would go.

CLOSE / A1: “For Assignment A1, submit a strategy, at least eight scored risks, at least twelve manual test cases, a completed exploratory charter, one evidence-backed defect report or explicitly marked simulated defect exercise, and a short explanation of how risks determine test levels. Commit your work to a branch and open a pull request. Our quality work begins with clear thinking; automation comes next.”

WEEK 2 — CLASS 1: Unit Testing, Fixtures and Assertions

Handbook: Module 3. **Teaching objective:** Students understand AAA, Vitest, matchers, isolated fixtures and coverage.

00:00–00:09 — Why we now automate

SAY: “Last week we listed risks and described test cases in English. This week we make some of those checks executable. Suppose we want to prove that adding two products priced twenty-five thousand naira each produces the correct total. We could open a browser, log in, click buttons and inspect the cart. But that is slow. If the actual risk is arithmetic, we should test the arithmetic function directly. That is unit testing. A unit test checks one small piece of logic with a predictable input and output, without requiring a server or browser.”

DEMO: Open `api/src/money.js` and `qa/tests/unit/money.test.js`. Explain why the import points to the real code being tested, not a copied function.

00:09–00:22 — Vitest and Arrange–Act–Assert

SAY: “Vitest is our JavaScript test runner. `describe` groups related checks. `it` gives one behaviour a readable name. `expect` states what the result should be. Good tests follow Arrange, Act, Assert. Arrange creates the input. Act calls the function. Assert compares the result to the requirement.

“Every test should have a clear reason to fail. Names like ‘works’ or ‘test one’ waste information. A useful name is ‘converts 19.99 naira to exactly 1999 kobo’. We use integer kobo because floating-point arithmetic can make currency wrong. In JavaScript, 0.1 plus 0.2 does not evaluate to exactly 0.3. Our tests therefore use `toBe` with exact integer values for money; we do not say ‘close enough’ when checking someone’s balance.”

DEMO: From `qa/`:

```
npm test -- tests/unit/money.test.js
```

SAY: “Now change one expected value in the test, for example 1999 to 2000. Run it again. You should see a red failure saying expected 2000 but received 1999. Restore it to 1999. Run once more: green. We deliberately broke only the test expectation to prove our assertion actually detects a difference. Do not edit the live product implementation to force green.”

00:22–00:33 — Matchers, fixtures and isolation

SAY: “We use different matchers for different questions. `toBe` compares primitive values exactly, so it is ideal for kobo integers. `toEqual` compares an array or object structure. `toMatchObject` checks selected fields without requiring us to predict every property. `toThrow` asserts that invalid input causes an error. For asynchronous operations, `await expect(promise).rejects.toThrow(...)` checks a rejected Promise.

“A fixture is prepared test state. Think of it like resetting a classroom desk before the next student arrives. If test A edits an array and test B inherits that changed array, test B may fail only in the full suite. `beforeEach` starts each test clean; `afterEach` restores changed state. `beforeAll` is for shared, safe and usually expensive setup. We will rely on fresh database fixtures next week.”

DEMO: Open `qa/tests/unit/validators.test.js`. Show `it.each` and say: “This is a table-driven test: each row becomes its own check. It prevents copy-paste mistakes and makes missing boundaries obvious.”

00:33–00:45 — Hands-on shipping threshold and coverage

SAY: “Our separate Week 2 `web/src/lib/cart.js` exercise introduces a free-shipping rule. This rule is not part of the live ShopDesk checkout; it is pure logic we practise on. Shipping costs 150,000 kobo — one thousand five hundred naira — when the subtotal is greater than zero and below five million kobo. At exactly five million

kobo, shipping becomes free. What three numbers should we test? Four million nine hundred ninety-nine thousand nine hundred ninety-nine, five million, and five million and one. An off-by-one error will show itself at the boundary."

DEMO: Run:

```
npm test -- tests/unit/cart.test.js
npm run test:coverage
```

SAY: "Coverage reports which source lines, statements, functions and branches were executed. Branch coverage tells us whether both sides of conditions have been exercised. Coverage does not prove that our assertions are correct, and 100% coverage is not automatically a good test suite. The starter project shows the mechanism. We move toward the handbook's eighty-percent coverage target as the course suite expands; our early starter does not pretend to have met the final gate."

00:45–00:55 — Student exercise

SAY: "Take five minutes to add two useful tests: one invalid email and one quantity just below the valid minimum. Then run the file and explain the failure messages to a partner. Next, add a boundary case to `cart.test.js`. Do not change the live storefront to get your expectation to pass. If there is disagreement between a requirement and the code, document it and ask which behaviour is intended."

CLOSE: "Next class we learn how to replace external dependencies with controlled stand-ins, how to freeze time and randomness, and how to demonstrate genuine test-driven development."

WEEK 2 — CLASS 2: Mocking, Isolation and TDD

Handbook: Module 4. **Teaching objective:** Students understand test doubles, `vi.fn`, `vi.spyOn`, `vi.mock`, fake timers, async failure paths and red–green–refactor.

00:00–00:10 — What makes tests unreliable

SAY: "Consider a function that confirms an order and then sends an email. If we make a real network request every time we test order logic, our result depends on internet speed, an email provider, account limits and possibly someone's inbox. We do not need the real provider to prove our code called it correctly. We isolate the code. We give it a stand-in called a test double.

"There are five useful names. A dummy is just a placeholder. A stub returns a planned answer. A spy records calls to a function, sometimes calling the real function. A mock is a controlled replacement whose interactions we verify. A fake is a small working version of something complicated, for example an in-memory data store. These names help explain what our tests are actually proving."

00:10–00:24 — vi.fn, spy, fake timer and asynchronous test

SAY: "Vitest gives us `vi.fn()` to create a function we control. We can tell it to return something, resolve a Promise, or reject with an error. We can assert the arguments it received and how many times it was called.

"A spy can replace a hidden input like randomness. Our `luckyDiscount` practice function uses `Math.random`. With real randomness it might return different outcomes on different runs. When we say `vi.spyOn(Math, 'random').mockReturnValue(0.05)`, the outcome is deterministic and testable. We must restore it after the test, otherwise other tests inherit the fake random number.

"Time is another hidden input. A new order ID may depend on the clock. We can freeze the clock with `vi.useFakeTimers()` and `vi.setSystemTime(...)`, then restore real time in `afterEach`. For asynchronous failures, we must await the expected rejection; if we forget `await`, the test may report success

before the work finishes."

DEMO: Open `qa/tests/unit/mocks.test.js`. Run:

```
npm test -- tests/unit/mocks.test.js
```

Review `trackEvent` assertions, mocked transport, `rejects.toThrow`, and time control.

00:24–00:33 — Dependency injection versus module mock

SAY: "Dependency injection sounds difficult, but the idea is simple: pass an external collaborator as an argument. Our practice checkout can receive a `trackEvent` function and a `now` function, allowing tests to control both without contacting any service. That is often cleaner than changing the module globally.

"When injection is inconvenient, `vi.mock` can replace a whole module. This is useful for analytics, HTTP and email boundaries. We should not mock the actual money-calculation function: then we would only prove our fake arithmetic works. We keep pure logic real and mock the risky external edge."

DEMO: Open `qa/tests/unit/module-mock.test.js` and run only that test file. Explain `toHaveBeenCalledWith`.

00:33–00:47 — Test-Driven Development live demonstration

SAY: "TDD means Test-Driven Development. We do three things: Red, Green, Refactor. Red: write a small test for behaviour that is not yet implemented. Run it and watch the right assertion fail. Green: implement the smallest change that makes it pass. Refactor: clean up the code without changing the behaviour, keeping the test green. Repeat. The test is a design tool as well as a check.

"Today our challenge is to add a `loyaltyPoints` function to the isolated practice module `web/src/lib/promo.js`. The rule is one point for every complete hundred naira spent. An order below one thousand naira earns no points. A single order can earn at most five hundred points. Notice that one thousand naira earns TEN points, not one. Reading requirements carefully is part of QA."

DEMO / true TDD (do not pre-create the implementation): Create a new temporary test file `qa/tests/unit/loyalty.test.js` with:

```
import { it, expect } from 'vitest';
import { loyaltyPoints } from '@web/lib/promo.js';

it('earns no points below one thousand naira', () => {
  expect(loyaltyPoints(99999)).toBe(0);
});
```

Run `npx vitest run tests/unit/loyalty.test.js` inside `qa/` and show the missing export failure. Then add this smallest practice function:

```
export function loyaltyPoints(totalKobo) {
  if (totalKobo < 100000) return 0;
  return Math.min(Math.floor(totalKobo / 10000), 500);
}
```

Run again. Add new test cases for exactly 100,000 kobo = 10 points; 5,000,000 = 500 points; 6,000,000 = 500 points; negative and fractional inputs should throw. Those last invalid-input cases will fail. Add validation only after observing the failure:

```
  if (!Number.isInteger(totalKobo) || totalKobo < 0) {
    throw new TypeError('totalKobo must be a non-negative integer');
  }
```

SAY: “Now you have an actual red–green sequence, not a screenshot of a test that happened to pass. Commit the test-first step before implementation whenever practicable. The TDD worksheet in `qa/test-plans/tdd-checkout.md` is where you record your red output, green output and one refactor.”

00:47–00:55 — Wrap and assignment A2

SAY: “Today we learned that good automation must be trustworthy. We isolate external services, avoid state leakage, restore spies and fake timers, assert asynchronous errors correctly, and let failing tests describe behaviour before code exists.

“For A2, submit money, validator and cart tests; demonstrate table-driven boundary values; show `vi.fn`, a spy, a module mock, fake timers and a rejected Promise; attach a coverage report; and preserve a genuine red–green–refactor journal. From Week 3 onward we will take the same thinking into Supertest, authenticated APIs and database transactions.”

Instructor quick reference

Date	Topic	File to open	Command / activity
Wed 7 Oct	Risk and strategy	`qa/test-plans/strategy.md`	Rank three risks
Sat 10 Oct	Manual and exploratory	`qa/test-plans/cases/catalog.md`	Browse / `curl` / bug report
Wed 14 Oct	Unit tests	`qa/tests/unit/`	`npm test -- tests/unit`
Sat 17 Oct	Mocking and TDD	`qa/tests/unit/mocks.test.js`	`npm test -- tests/unit/mocks.test.js`

What to correct explicitly when teaching from the handbook

The handbook contains **inconsistent examples** across its setup and later weekly chapters. The provided application specification uses `price_kobo`, `quantity`, `createDb`, validation HTTP 400, `error.fields` and a simple products/search/orders API. Later Week 2/3 pages demonstrate `priceKobo`, `qty`, stricter frontend password validation, shipping/coupons, HTTP 422, and additional routes. Treat those later functions as **separate learning exercises** unless the application is explicitly expanded, not live functionality students must falsely mark as passing. The downloaded codebase and starter tests follow one clear contract and explicitly label practice-only helpers.

If learners uncover an actual defect, **record it** with steps and evidence. Do not edit the SUT just to turn the check green. The isolated `web/src/lib/` TDD practice area is the deliberate exception for implementing a new learning exercise.